

Full Object-Oriented Programming (OOP) Concept in C++

1. Introduction to OOP

Object-Oriented Programming (OOP) is a **programming paradigm** where software is designed using **objects**.

An **object** represents a real-world entity that has:

- **Attributes** → Data or properties (variables).
- **Behaviors** → Functions or actions (methods).

Why OOP?

- Organizes code better than procedural programming.
- Improves **modularity, reusability, and maintainability**.
- Suitable for **complex and large-scale projects**.

2. Key Principles of OOP

2.1 Encapsulation

Definition: Bundling **data** and the **functions** that operate on that data into a single unit (class) and restricting direct access to the internal representation.

How to Achieve:

- Use **access specifiers**:
 - **private**: Accessible only within the class.
 - **protected**: Accessible within the class and derived classes.
 - **public**: Accessible from anywhere.

Advantages:

- Protects data from unintended changes.
- Allows **controlled access** via getter/setter functions.



```
1 #include <iostream>
2 using namespace std;
3
4 class BankAccount {
5 private:
6     double balance; // hidden data
7
8 public:
9     BankAccount(double initialBalance) {
10         balance = initialBalance;
11     }
12
13     void deposit(double amount) {
14         if(amount > 0)
15             balance += amount;
16     }
17
18     double getBalance() const { // controlled access
19         return balance;
20     }
21 };
22
23 int main() {
24     BankAccount account(1000);
25     account.deposit(500);
26     cout << "Balance: $" << account.getBalance();
27     // Output: Balance: $1500
28 }
```

2.2 Abstraction

Definition: Hiding internal implementation details and showing only the necessary information to the outside world.

How to Achieve:

- Use **abstract classes** (classes with pure virtual functions).
- Use **interfaces** (in C++, achieved with only pure virtual functions).

Advantages:

- Reduces complexity for the user.
- Improves focus on what an object does rather than how.

```
1 #include <iostream>
2 using namespace std;
3
4 class Shape {
5 public:
6     virtual void draw() = 0; // pure virtual function
7 };
8
9 class Circle : public Shape {
10 public:
11     void draw() override {
12         cout << "Drawing Circle" << endl;
13     }
14 };
15
16 class Rectangle : public Shape {
17 public:
18     void draw() override {
19         cout << "Drawing Rectangle" << endl;
20     }
21 };
22
23 int main() {
24     Shape* s = new Circle();
25     s->draw(); // "Drawing Circle"
26     delete s;
27 }
28
```

2.3 Inheritance

Definition: A mechanism by which a new class (derived) acquires properties and behaviors of an existing class (base).

Types:

1. **Single** → One base, one derived.
2. **Multiple** → Multiple bases, one derived.
3. **Multilevel** → Derived class becomes base for another derived class.
4. **Hierarchical** → One base class with multiple derived classes.
5. **Hybrid** → Combination of types.

Advantages:

- Promotes **code reusability**.
- Supports **extensibility**.

```
1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     void start() { cout << "Vehicle started\n"; }
7 };
8
9 class Car : public Vehicle {
10 public:
11     void honk() { cout << "Car honks!\n"; }
12 };
13
14 int main() {
15     Car c;
16     c.start(); // from Vehicle
17     c.honk();  // from Car
18 }
19
```

2.4 Polymorphism

Definition: Ability of the same interface to represent different underlying forms (data types or behaviors).

Types in C++:

1. Compile-time (Static) Polymorphism

- Function overloading
- Operator overloading

2. Run-time (Dynamic) Polymorphism

- Virtual functions
- Function overriding

```
1 #include <iostream>
2 using namespace std;
3
4 class Animal {
5 public:
6     virtual void sound() { cout << "Some animal sound\n"; }
7 };
8
9 class Dog : public Animal {
10 public:
11     void sound() override { cout << "Woof!\n"; }
12 };
13
14 class Cat : public Animal {
15 public:
16     void sound() override { cout << "Meow!\n"; }
17 };
18
19 int main() {
20     Animal* a = new Dog();
21     a->sound(); // "Woof!"
22     delete a;
23 }
24
```

3. Other Important OOP Concepts in C++

3.1 Class vs Object

- **Class:** Blueprint/template for creating objects.
- **Object:** Instance of a class.

3.2 Constructor & Destructor

- **Constructor:** Special method called automatically when an object is created.
- **Destructor:** Special method called automatically when an object is destroyed.

3.3 Friend Function

- Allows an external function to access private/protected members of a class.

```
1 class Box {
2 private:
3     int width;
4 public:
5     Box(int w) : width(w) {}
6     friend void showWidth(Box b);
7 };
8
9 void showWidth(Box b) {
10     cout << "Width: " << b.width << endl;
11 }
12
```

3.4 Operator Overloading

- Redefines how operators work for user-defined types.

```
1 class Complex {
2 public:
3     int real, imag;
4     Complex(int r=0, int i=0) : real(r), imag(i) {}
5
6     Complex operator + (const Complex& c) {
7         return Complex(real + c.real, imag + c.imag);
8     }
9 };
10
```

3.5 this Pointer

- Refers to the calling object's address.

```
1 class Demo {
2 public:
3     void show() { cout << "Object address: " << this <<
4         endl; }
5 };
```

3.6 Static Members

- Belong to the class, not individual objects.

```
1 class Counter {
2 public:
3     static int count;
4     Counter() { count++; }
5 };
6 int Counter::count = 0;
7
```

3.7 Virtual Destructor

- Ensures derived class destructors are called when deleting via a base pointer.

```
1 class Base {
2 public:
3     virtual ~Base() { cout << "Base Destructor\n"; }
4 };
5
```

4. Advantages of OOP

- Modularity
- Reusability
- Scalability
- Maintainability
- Security through encapsulation

5. Disadvantages of OOP

- Slightly higher memory usage.
 - Requires more upfront design effort.
 - Can be complex for small programs.
-

6. Recap Table

Concept	Purpose	C++ Feature Used
Encapsulation	Data protection	Access specifiers
Abstraction	Hide complexity	Pure virtual functions
Inheritance	Code reusability	Base & derived classes
Polymorphism	Flexible behavior	Virtual functions, overloading